



Blaise Izerimana

Fida Ukwishaka

Remy Nteziryayo

Group 15 Assignment 2

Department of Computer Sciences and Information Systems, Ashesi University

CS456: Algorithm Design and Analysis

Sampson Asare

October 2, 2026

Question 1 Network Topology Classification

Definitions used by the algorithm

- **Ring:** Every vertex has degree 2. Therefore, every row of the adjacency matrix contains exactly two 1.
- **Star:** Exactly one vertex has degree $n - 1$, while every other vertex has degree 1.
- **Fully connected mesh:** Every vertex is connected to every vertex. Thus $A[i][j] = 1$ for every $i \neq j$, and $A[i][i] = 0$. Meaning a vertex can not connect to itself

Brute-force algorithm

1. Input the $n \times n$ Boolean adjacency matrix A .
2. Check that $A[i][i] = 0$ for every vertex and that $A[i][j] = A[j][i]$ for all i, j . If not, return "None of the three topologies."
3. Compute the degree of every vertex by counting the 1s in each row.
4. Check for a fully connected mesh: every off-diagonal entry is 1.
5. Check for a star: exactly one degree is $n - 1$ and every other degree is 1.
6. Check for a ring: every vertex has degree 2.
7. If one test pass, show which topology. Otherwise report "None."

Pseudocode

ALGORITHM CLASSIFY-TOPOLOGY(A, n)

```
    for  $i \leftarrow 0$  to  $n - 1$ 
        if  $A[i][i] = 1$ 
            return NONE
        degree[i]  $\leftarrow 0$ 
        for  $j \leftarrow 0$  to  $n - 1$ 
            degree[i]  $\leftarrow$  degree[i] +  $A[i][j]$ 

    for  $i \leftarrow 0$  to  $n - 1$ 
        for  $j \leftarrow i + 1$  to  $n - 1$ 
            if  $A[i][j] \neq A[j][i]$ 
                return NONE

    mesh  $\leftarrow$  TRUE
    for  $i \leftarrow 0$  to  $n - 1$ 
        for  $j \leftarrow 0$  to  $n - 1$ 
            if  $i \neq j$  AND  $A[i][j] \neq 1$ 
                mesh  $\leftarrow$  FALSE

    if mesh = TRUE
        return FULLY_CONNECTED_MESH

    centerCount  $\leftarrow 0$ 
```

```

leafCount ← 0
for i ← 0 to n - 1
    if degree[i] = n - 1
        centerCount ← centerCount + 1
    else if degree[i] = 1
        leafCount ← leafCount + 1

if centerCount = 1 AND leafCount = n - 1
    return STAR

ring ← TRUE
for i ← 0 to n - 1
    if degree[i] ≠ 2
        ring ← FALSE

if ring = TRUE
    return RING

return NONE

```

This is how our algorithm works in words

The algorithm looks at the connections between the nodes so that we can find the type of the network

- **Fully connected mesh:** Every vertex is connected to every other vertex. So, we check all the connections in the matrix.
- **Star:** One vertex is connected to all the other vertices. The other vertices are connected only to the center. The degrees are $n - 1$ this is for the center node, 1, 1, ..., 1. The rest ones are for the outer
- **Ring:** Every vertex is connected to exactly two other vertices. So, every vertex has degree 2

If the matrix matches one of these conditions, we know the type of topology. If it does not match any of them, then we return none

Time efficiency

The matrix has n^2 entries. Finding the degree of each vertex takes $\Theta(n^2)$ time. Checking the symmetry and checking for mesh also take $\Theta(n^2)$ time. The star and ring checks take $\Theta(n)$ time. Therefore, the total running time is $\Theta(n^2)$.

Using summation method:

$$T(n) = \sum_{i=1}^n (n - 1) = n(n - 1) = n^2 - n = \Theta(n^2)$$

Question 2: Alternating Disks

The initial arrangement is D, L, D, L, ..., D, L, where D represents a dark disk and L represents a light disk. The required final arrangement is L, L, ..., L, D, D, ..., D. Only adjacent disks may be interchanged.

(a) Algorithm

We used an adjacent-swap procedure similar to bubble sort. We scan from left to right. Whenever a dark disk is immediately followed by a light disk (D, L), swap the two disks. Continue scanning until no such adjacent pair remains. At that point all light disks are to the left of all dark disks.

ALGORITHM ALTERNATING-DISKS(n)

```
disks  $\leftarrow$  [D, L, D, L, ..., D, L]      // 2n disks
moves  $\leftarrow$  0
```

```
repeat
  swapped  $\leftarrow$  FALSE
```

```
  for i  $\leftarrow$  0 to 2n - 2
    if disks[i] = D AND disks[i + 1] = L
      swap(disks[i], disks[i + 1])
      moves  $\leftarrow$  moves + 1
      swapped  $\leftarrow$  TRUE
```

```
until swapped = FALSE
```

```
return disks, moves
```

Number of moves

We have n dark disks and n light disks. Initially, every dark disk is to the left of every light disk. To reach the required arrangement, each dark-light pair must reverse its relative order. There are $n \times n = n^2$ such pairs, and an adjacent swap can reverse the order of only one such pair. Hence at least n^2 moves are necessary. The algorithm performs exactly one adjacent swap for each dark-light inversion, so it uses exactly:

$$\text{Number of moves} = n^2$$

For the example shown in the assignment, $n = 4$, so the minimum number of moves is $4^2 = 16$.

Why the algorithm terminates with the required arrangement

Every swap changes a D,L pair into L,D, so the number of dark-light inversions decreases by exactly one. The number of inversions cannot become negative. Therefore the process eventually stops. When no D,L adjacent pair remains, there cannot be a light disk to the right of a dark disk; otherwise the boundary between the two groups would contain a D,L pair. Thus all light disks are on the left and all dark disks are on the right.

Time efficiency

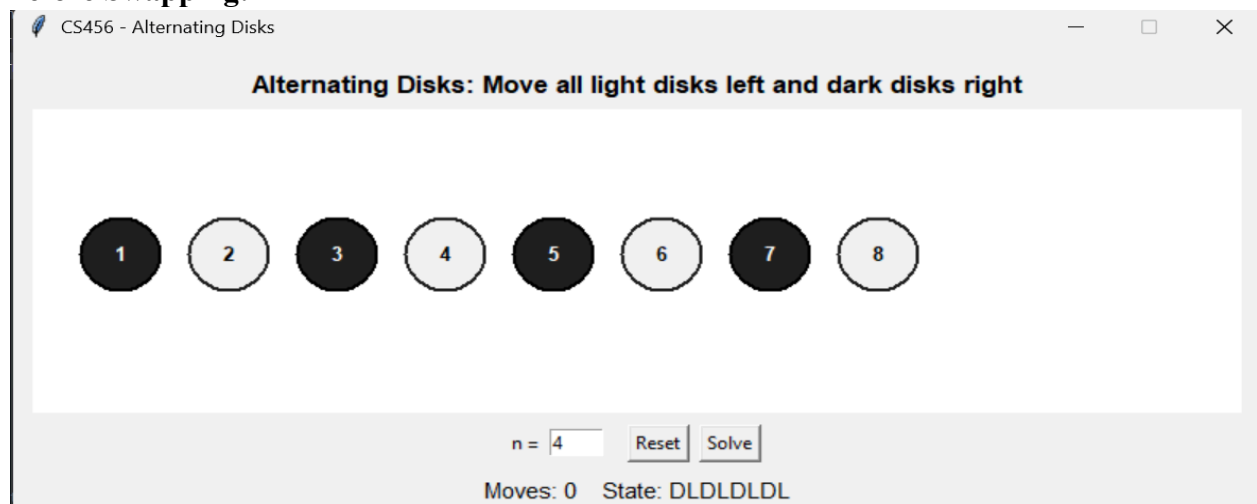
The algorithm performs n^2 swaps. With a straightforward adjacent-scan implementation, each pass takes $O(n)$ and there are $O(n)$ passes, giving $O(n^2)$ time in the worst case. The number of swaps itself is exactly n^2 .

(b) Python program: visual simulation

The accompanying Python program, “alternating_disks_visual.py” we created provides a Tkinter graphical interface. It displays the disks as circles, highlights each pair being swapped, updates the move counter, and shows the current disk order. As a default example, we used $n = 4$, but another positive value of n can be entered.

We used only the Python's standard Tkinter library, so that we don't have to install any other external packages.

Before Swapping:



After Swapping:

